PRODUCT REQUIREMENTS DOCUMENT

Email Engagement Dashboard

Mailchimp-style intelligence layer for The Horeca Meetup admin

Version 2.0 (FINAL) · May 1, 2026

Prepared for: Cursor Agent · Prepared by: Noman Peera

Repos: NomanPeera-Horeca/horeca-events + horeca-admin

Supabase project: hgvfixbopldwptnfpxwc

> **VISUAL REFERENCE** This PRD references admin-dashboard-v3.html (the approved interactive mockup). Cursor MUST open and review the mockup file as the visual source of truth before implementing. Match the design pixel-by-pixel.

1. Executive Summary

This document specifies the build of an email engagement dashboard for The Horeca Meetup admin platform. The system tracks every email event from Resend (sent, delivered, opened, clicked, bounced, forwarded signals) at the per-recipient level, displays results in a Mailchimp-style dashboard with inline expandable campaigns, and surfaces forensic forwarding signals based on multi-city/multi-device open patterns.

The strategic outcome: before the May 5 event door opens, the admin sees at-a-glance which guests are highly engaged (multiple opens, calendar adds), which are passive (delivered but never opened), which forwarded the email to colleagues (likely +1 candidates), and which had bad email addresses (need alternate outreach). All visible inline without drilling into individual cards.

> **KEY DESIGN PRINCIPLE** Mailchimp dashboard pattern: everything visible at-a-glance. Click a campaign bar → expands inline to show all recipients with their per-email engagement. No drawer, no modal, no separate detail page. The data IS the interface.

2. Business Context

2.1 Strategic positioning

The Horeca Meetup is positioning itself as the Aspire/Inman/SaaStr of hospitality. Vol. II (May 5, Houston) targets 100 restaurant and hotel operators. The admin platform must signal "serious series," not "local meetup." Engagement intelligence is what separates Eventbrite Pro from a Google Form.

2.2 User

Single primary user (Noman Peera) plus a small team operating from mobile during pre-event week. The dashboard is responsive but optimized for desktop where pre-event analysis happens.

2.3 Success metrics

- Day -3 to event: identify cold guests via filter, send personal WhatsApp nudges
- Day -1: pull "hot guest" list (high opens + clicked + cal added) for door staff
- Day +1: identify likely-forwarded campaigns to invite +1 conversions for Vol. III
- Vol. III planning: identify "ghost" guests (never opened any email) and exclude

3. Current State (Already Built)

The following exists and works in production. Do not modify unless explicitly listed in Section 5:

- Public registration site at events.thehorecastore.com (AWS Amplify)
- Admin panel at horeca-admin.onrender.com (Render)
- Supabase project hgvfixbopldwptnfpxwc with tables: registrations, events, email_templates (11 rows), admin_users, activity_log
- Edge Function send-registration-email v3 deployed via Supabase CLI
- Resend integration sending from events@thehorecastore.com (domain verified)
- Inline base64 QR codes in approval emails (Outlook-proof)
- Calendar URLs (Google + ICS) in approval and approval-plus-one templates
- VIP track: vip-apply.html, is_vip column, approval-vip template
- Check-in flow: checkin.html with admin auth gate

4. Scope

4.1 In scope

- New Edge Function: resend-webhook (receives engagement events from Resend)
- New database table: email_events (per-event log with location and device data)
- New columns on registrations table (per-email aggregate metrics)
- New view: v_recipient_engagement (powers dashboard table)
- Update Edge Function send-registration-email to tag emails with registration_id
- Admin panel: replace existing engagement view with new Mailchimp-style dashboard
- Implementation must match admin-dashboard-v3.html visually
- Per-campaign expandable accordion with inline recipient table
- Forwarded detection logic (city/IP/device-based forensic signals)
- Filter pills inside expanded campaign (All / Opened / Not Opened / Clicked / Forwarded / Bounced)
- Real-time updates via Supabase Realtime

4.2 Out of scope (Phase 2)

- SMS engagement tracking

- Auto-send WhatsApp nudge to cold guests (UI button stub only)
- Cohort comparison across events (Vol. II vs Vol. III)
- A/B testing of email subject lines
- Predictive no-show scoring

5. Dashboard UI Specification

> **VISUAL SOURCE OF TRUTH** admin-dashboard-v3.html is the approved design. Cursor must open this file and match the design exactly: typography (Fraunces serif + Inter sans + JetBrains Mono), color palette (luxury black/gold/cream), spacing, hover states, and motion. Do not redesign.

5.1 Page structure

Three vertical zones from top to bottom:

- Top stats strip: 6 cells showing aggregate metrics across ALL campaigns
- Hint banner: instruction text reminding admin to click campaigns to expand
- Campaign accordion: 5 collapsible campaign cards (1 sent + 4 scheduled for Vol. II)

5.2 Top stats strip (6 cells)

Background: var(--bg-elevated). Border-radius: 10px. Single horizontal row with 1px gold dividers between cells.

Cell	Label	Number Color	Meta
1	Sent	White	total recipients
2	Delivered	Green	__% delivery rate
3	Opened	Gold	__% open rate
4	Clicked	Gold	__% click rate
5	Forwarded	Gold	__% likely shared
6	Bounced	Red	__% needs cleanup

5.3 Campaign accordion

Each campaign is a card with two sections: a campaign bar (always visible) and an expandable recipient table (visible when expanded).

Campaign bar (always visible)

Three-column grid: chevron icon, campaign info (name + metadata), campaign stats (5 percentage stats).

- Chevron rotates 90° when expanded

- Icon: 📧 for sent campaigns (gold soft background), ⏸️ for scheduled (dim background)
- Name: 18px Fraunces serif
- Metadata line (JetBrains Mono 11px): "Sent Apr 30 · 4:00 PM · 47 recipients · ● Live · tracking"
- Live indicator (gold) shown only for sent campaigns
- Stats column: Delivered % / Opened % / Clicked % / Cal Add % / Fwd % / Bounced %
- Bounced stat in red, all others in gold for sent campaigns, dim dash for scheduled

Expanded recipient table

Visible only when campaign is expanded. Contains four sections:

- Toolbar (top): filter pills + action buttons
- Header row: column headers in JetBrains Mono 9px uppercase
- Body rows: zebra striping, hover state, per-recipient data
- Bulk bar (bottom): showing-count + bulk action buttons

5.4 Recipient table columns (left to right)

Column	Width	Content	Visual
Recipient	flexible	Name (white) + email (gray mono small)	two lines
Business	flexible	Company name + role/title	two lines
Delivered	narrow	Green ✓ if delivered, red ✗ if bounced, dim — if pending	circular badge
Opened	narrow	Green ✓ if at least 1 open, dim — if not opened	circular badge
Times	narrow	Total open count (4 = opened 4 times)	gold pill if >0, dim if 0
Clicked	narrow	Total click count	gold pill if >0, dim if 0
Cal Add	narrow	Green ✓ if calendar link clicked, dim — otherwise	circular badge
Fwd	narrow	↗ YES (orange) / ↗ MAYBE (blue) / — (dim)	pill — see Section 6
Last Activity	medium	Relative time of most recent event	gold if recent (<6hr), tertiary otherwise
Action	narrow	Button: VIEW / NUDGE / FIX EMAIL based on state	small button

5.5 Filter pills (toolbar inside expanded campaign)

Below campaign bar, above table. Horizontal row of clickable pills:

- All __ (default active, white text on gold)
- Opened __ (filters to recipients who opened)
- Not Opened __ (delivered but not opened)
- Clicked __ (clicked any link)
- Forwarded __ (likely or maybe forwarded)
- Bounced __ (delivery failed)

Plus right-aligned: "Export CSV" + "Resend to Cold" buttons.

5.6 Row action buttons (rightmost column)

Button label changes by recipient state:

State	Button Label	Style	On Click
Engaged	VIEW	outline	Open recipient detail modal
Cold (no opens)	NUDGE	gold filled	Open WhatsApp message composer
Bounced	FIX EMAIL	outline	Open email correction modal

5.7 Bulk bar (bottom of expanded campaign)

Shows pagination info + bulk action buttons:

- Left: "Showing 12 of 47 recipients · scroll for more"
- Right: "Send WhatsApp to Cold (3)" button
- Right: "Open Full View" button (opens dedicated full-page table)

6. Forwarded Detection Logic

> **IMPORTANT** Email systems do not provide deterministic forward detection. SMTP and email tracking pixels cannot reliably tell us if an email was forwarded. The "Fwd" column shows forensic signal-based likelihood, NOT precise forward count. Mailchimp and HubSpot have the same limitation.

6.1 Detection algorithm

On every "opened" event from Resend, capture the IP address, user agent, and city (from Resend payload). Store in email_events table. Then evaluate aggregate signals across all opens for that recipient + email combination:

```
-- Pseudo-logic for forwarded detection

FOR each (registration_id, email_template_slug) combination:
  unique_cities      = COUNT DISTINCT city FROM email_events WHERE
event_type='opened'
  unique_ips         = COUNT DISTINCT ip_address FROM email_events WHERE
event_type='opened'
  unique_user_agents = COUNT DISTINCT user_agent FROM email_events WHERE
event_type='opened'

  IF unique_cities >= 2 OR unique_ips >= 3:
     → forwarded_status = 'yes'  (high confidence)
     → display: ↗ YES (orange pill)

  ELIF unique_user_agents >= 3:
     → forwarded_status = 'maybe' (soft signal)
     → display: ↗ MAYBE (blue pill)

  ELSE:
     → forwarded_status = null
     → display: — (dim dash)
```

6.2 Why this approach is sound

- A real recipient (Bilal in Houston) opening from his iPhone + laptop = same city, same IP range = NOT forwarded

- Bilal forwards to Tanveer in Lahore = open from 2 cities = strong forward signal
- Bilal forwards to colleague in same office = different IP, same city = soft signal
- Multiple devices but same city + same IP = single user, NOT a forward

6.3 What the column does NOT show

- Exact count of how many people received the forward
- Email addresses of people who received the forward
- Whether the forward action was explicitly taken (vs. shared via screenshot)

This information is fundamentally unavailable via SMTP. Do not promise it.

7. Data Model

7.1 New table: email_events

One row per webhook event from Resend. Stores everything needed for both per-event display and forensic forwarding analysis.

```
CREATE TABLE email_events (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  registration_id       UUID REFERENCES registrations(id) ON DELETE CASCADE,
  resend_email_id       TEXT NOT NULL,
  email_template_slug   TEXT NOT NULL,    -- approval, reminder-1week, etc.
  event_type            TEXT NOT NULL,    -- sent, delivered, opened, clicked,
bounced
  recipient_type        TEXT,             -- main | plus_one
  recipient_email       TEXT,
  occurred_at           TIMESTAMPTZ NOT NULL,
  link_url              TEXT,             -- for clicked events
  ip_address            INET,             -- for forwarding detection
  user_agent            TEXT,             -- for forwarding detection
  city                  TEXT,             -- from Resend payload
  country               TEXT,
  raw_payload           JSONB,
  created_at            TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_ee_registration ON email_events(registration_id);
CREATE INDEX idx_ee_template ON email_events(email_template_slug);
CREATE INDEX idx_ee_type ON email_events(event_type);
CREATE INDEX idx_ee_occurred ON email_events(occurred_at DESC);
CREATE UNIQUE INDEX idx_ee_dedup ON email_events(
  resend_email_id, event_type, occurred_at
);  -- prevents duplicate webhook processing
```

7.2 New columns on registrations

Aggregate counters per email template, computed on every webhook event for fast dashboard queries.

```
ALTER TABLE registrations
  ADD COLUMN IF NOT EXISTS email_engagement JSONB DEFAULT '{}';

-- Structure of email_engagement JSONB:
-- {
--   "approval": {
--     "sent_at": "2026-04-30T16:00:00Z",
--     "delivered": true,
--     "opened": true,
--     "open_count": 4,
--     "first_opened_at": "2026-04-30T16:03:00Z",
--     "last_opened_at": "2026-05-01T08:15:00Z",
--     "click_count": 2,
--     "calendar_added": true,
--     "bounced": false,
--     "forwarded_status": "yes",  // yes | maybe | null
--     "unique_cities": 2,
--     "unique_ips": 3,
--     "unique_devices": 4
--   },
--   "reminder-1week": { ... },
--   "reminder-day-of": { ... }
-- }
```

7.3 New view: v_recipient_engagement

Powers the expanded recipient table for any campaign. One row per (registration, email) pair.

```
CREATE VIEW v_recipient_engagement AS
SELECT
  r.id              AS registration_id,
  r.full_name,
  r.email,
  r.business_name,
  r.role_title,
  r.event_id,
  ee.email_template_slug,
  bool_or(ee.event_type = 'delivered') AS delivered,
  bool_or(ee.event_type = 'opened')    AS opened,
  COUNT(*) FILTER (WHERE ee.event_type = 'opened')  AS open_count,
  COUNT(*) FILTER (WHERE ee.event_type = 'clicked') AS click_count,
  bool_or(ee.event_type = 'bounced')   AS bounced,
  bool_or(ee.link_url ILIKE '%calendar.google.com%'
       OR ee.link_url ILIKE '%/calendar.html%') AS calendar_added,
  COUNT(DISTINCT ee.city)       FILTER (WHERE ee.event_type = 'opened') AS
unique_cities,
  COUNT(DISTINCT ee.ip_address) FILTER (WHERE ee.event_type = 'opened') AS
unique_ips,
  COUNT(DISTINCT ee.user_agent) FILTER (WHERE ee.event_type = 'opened') AS
unique_devices,
  MAX(ee.occurred_at) AS last_activity,
  CASE
    WHEN COUNT(DISTINCT ee.city) FILTER (WHERE ee.event_type = 'opened') >= 2
THEN 'yes'
    WHEN COUNT(DISTINCT ee.ip_address) FILTER (WHERE ee.event_type = 'opened')
>= 3 THEN 'yes'
    WHEN COUNT(DISTINCT ee.user_agent) FILTER (WHERE ee.event_type = 'opened')
>= 3 THEN 'maybe'
    ELSE NULL
  END AS forwarded_status
FROM registrations r
LEFT JOIN email_events ee ON ee.registration_id = r.id
WHERE ee.email_template_slug IS NOT NULL
GROUP BY r.id, ee.email_template_slug;
```

7.4 New view: v_campaign_summary

Powers the campaign bar stats (top right of each campaign card).

```
CREATE VIEW v_campaign_summary AS
SELECT
  email_template_slug,
  COUNT(DISTINCT registration_id)                                AS sent_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE delivered)        AS delivered_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE opened)           AS opened_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE click_count > 0)  AS clicked_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE calendar_added)   AS cal_added_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE forwarded_status IS NOT NULL) AS forwarded_count,
  COUNT(DISTINCT registration_id) FILTER (WHERE bounced)          AS bounced_count
FROM v_recipient_engagement
GROUP BY email_template_slug;
```

8. Edge Functions

8.1 New: resend-webhook

Receives webhook POSTs from Resend. Verifies signature. Inserts row into email_events. Updates registrations.email_engagement JSONB.

Path: supabase/functions/resend-webhook/index.ts

Webhook signature verification (svix)

Resend uses svix for webhook signing. Verify the svix-signature header matches HMAC-SHA256 of svix-id + svix-timestamp + body, signed with RESEND_WEBHOOK_SECRET.

```
import { Webhook } from 'https://esm.sh/svix@1.15.0';

const secret = Deno.env.get('RESEND_WEBHOOK_SECRET')!;
const wh = new Webhook(secret);
const headers = {
  'svix-id': req.headers.get('svix-id')!,
  'svix-timestamp': req.headers.get('svix-timestamp')!,
  'svix-signature': req.headers.get('svix-signature')!,
};

let event;
try {
  event = wh.verify(await req.text(), headers);
} catch (err) {
  return new Response('Invalid signature', { status: 401 });
}
```

Event handling

Map Resend event types and persist:

Resend Event	Action
email.sent	Insert email_events row, update email_engagement.{slug}.sent_at
email.delivered	Insert row, update email_engagement.{slug}.delivered = true

email.opened	Insert row with city/IP/UA, update open_count + recompute forwarded_status
email.clicked	Insert row with link_url, update click_count + check if calendar link
email.bounced	Insert row, update email_engagement.{slug}.bounced = true
email.complained	Insert row, treat as terminal bounced

> **CRITICAL** Always return 200 OK to Resend, even on internal errors. Resend retries failed webhooks aggressively. If the function returns 500, you get duplicate events. Log errors internally but acknowledge the webhook with 200.

8.2 Update: send-registration-email

Add registration_id and template_slug to Resend tags so webhook events can be attributed.

```
await fetch('https://api.resend.com/emails', {
  method: 'POST',
  body: JSON.stringify({
    from, to, subject, html,
    tags: [
      { name: 'registration_id', value: registration.id },
      { name: 'event_slug',      value: event.slug },
      { name: 'template_slug',   value: template.slug },
      { name: 'recipient_type',  value: 'main' },  // or plus_one
    ]
  })
});
```

9. Frontend Implementation

9.1 File location

Add the engagement view to admin.html in the horeca-admin repo. Replace any existing engagement-related code.

9.2 Reference file

admin-dashboard-v3.html is the visual source of truth. All HTML structure, CSS classes, design tokens (CSS variables), animations, and hover states should match exactly. Cursor must view this file before writing code.

9.3 Component breakdown

9.3.1 Top stats strip

Static structure. Wire up data from v_campaign_summary aggregated across all sent campaigns.

```
SELECT
  SUM(sent_count) AS total_sent,
  SUM(delivered_count) AS total_delivered,
  SUM(opened_count) AS total_opened,
  SUM(clicked_count) AS total_clicked,
  SUM(forwarded_count) AS total_forwarded,
  SUM(bounced_count) AS total_bounced
FROM v_campaign_summary
WHERE email_template_slug IN (
  SELECT slug FROM email_templates WHERE event_id = $current_event_id
);
```

9.3.2 Campaign accordion

Loop through email templates for the current event. For each template:

- Render campaign bar with name, sent date, recipient count, live/scheduled status
- If template has been sent → show campaign stats from v_campaign_summary
- If template not yet sent → show dim dashes for stats
- Bind click handler on bar to toggle .expanded class on parent .campaign div

- Lazy-load recipient table data on first expand (avoid loading all 5 campaigns × 47 rows on page load)

9.3.3 Recipient table

When a campaign expands, fetch v_recipient_engagement WHERE email_template_slug = current template. Render rows according to column spec in Section 5.4.

```
// Fetch on expand
const { data } = await supabase
  .from('v_recipient_engagement')
  .select('*')
  .eq('email_template_slug', currentTemplate)
  .eq('event_id', currentEventId)
  .order('open_count', { ascending: false });
```

9.3.4 Filter pills

Client-side filter (do not refetch). On pill click:

- Update active class
- Filter loaded data array
- Re-render visible rows

9.3.5 Action buttons

Button label and behavior depend on row state. See Section 5.6.

```
function getActionButton(row) {
  if (row.bounced) return { label: 'FIX EMAIL', onClick: openEmailFixModal };
  if (!row.opened) return { label: 'NUDGE', onClick: openWhatsAppComposer,
primary: true };
  return { label: 'VIEW', onClick: openRecipientDetailModal };
}
```

9.4 Real-time updates

Subscribe to email_events INSERT via Supabase Realtime. On every event:

- Identify which (registration_id, email_template_slug) was affected
- Refetch v_recipient_engagement for that pair only
- Update the corresponding row in the DOM (find by data-registration-id attribute)

- Animate gold pulse on changed cells (600ms ease-out)
- Update campaign-level stats by refetching v_campaign_summary for that template

```
const channel = supabase
  .channel('email_events')
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'email_events' },
    async (payload) => {
      const { registration_id, email_template_slug } = payload.new;
      await refreshRecipientRow(registration_id, email_template_slug);
      await refreshCampaignStats(email_template_slug);
    })
  .subscribe();
```

10. Deployment Steps

Execute in this exact order. All commands run from the Cursor terminal in /Users/nomanpeera/Downloads/Horeca_Events. Cursor has already run supabase login.

Step 1: Create migration file

Path: database/email_engagement_dashboard.sql

Content: schema from Section 7 (table + indexes + 2 views + ALTER TABLE).

Step 2: Run migration

```
supabase db query --linked < database/email_engagement_dashboard.sql
```

Step 3: Create webhook function

Path: supabase/functions/resend-webhook/index.ts

Content: webhook handler from Section 8.1 with svix verification.

Step 4: Deploy webhook function

```
supabase functions deploy resend-webhook --no-verify-jwt
```

Step 5: Update send-registration-email

Modify Resend API call to include tags array per Section 8.2.

```
supabase functions deploy send-registration-email --no-verify-jwt
```

Step 6: Configure Resend webhook

- Resend dashboard → Webhooks → Add Endpoint
- URL: https://hgvfixbopldwptnfpxwc.supabase.co/functions/v1/resend-webhook
- Events: email.sent, email.delivered, email.opened, email.clicked, email.bounced, email.complained, email.delivery_delayed
- Copy the signing secret (starts with whsec_)

Step 7: Set webhook secret

```
supabase secrets set RESEND_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxx
```

Step 8: Enable Realtime

Supabase Dashboard → Database → Replication → toggle ON for email_events table.

Step 9: Update admin.html

Replace existing engagement view (if any) with the new dashboard. Match admin-dashboard-v3.html visually. Wire to new views from Section 7.3 and 7.4.

```
cd /Users/nomanpeera/Downloads/Horeca_Admin
git add admin.html
git commit -m "feat: Mailchimp-style engagement dashboard"
git push origin main  # Render auto-deploys
```

Step 10: Verification test

- Submit a fresh test registration on events.thehorecastore.com
- Approve from admin → triggers send-registration-email
- Open the email → wait 30 sec → check email_events table for opened row
- Click "Add to Google Calendar" → wait → check calendar_added in v_recipient_engagement
- Confirm row appears in admin dashboard with correct counts
- Confirm real-time pulse animation triggers on new event arrival

11. Acceptance Criteria

Build is complete when ALL pass:

Database

- email_events table created with all columns and indexes from Section 7.1
- registrations.email_engagement column added (JSONB)
- v_recipient_engagement view returns rows correctly
- v_campaign_summary view returns rows correctly
- Forwarded status logic per Section 6.1 returns correct values

Edge functions

- resend-webhook deployed and reachable
- Test POST with valid svix signature returns 200
- Test POST with invalid signature returns 401
- send-registration-email tags every send with registration_id and template_slug

Data flow

- Sending an approval email creates email_events.event_type=sent within 30 sec
- Opening the email creates event_type=opened within 30 sec

- Clicking calendar link creates event_type=clicked AND v_recipient_engagement.calendar_added = true
- Bouncing creates event_type=bounced AND email_engagement.bounced = true
- Forwarded detection: opening from 2 distinct cities sets forwarded_status = yes

Dashboard UI

- Top stats strip displays 6 cells with correct numbers
- All email template campaigns appear as collapsible cards
- Sent campaigns show campaign stats; scheduled show dim dashes
- Clicking a campaign bar expands it inline (no modal, no drawer)
- Expanded recipient table shows all 10 columns from Section 5.4
- Filter pills work correctly (client-side filter)
- VIEW / NUDGE / FIX EMAIL buttons render based on recipient state
- Forwarded column shows ↗ YES (orange) / ↗ MAYBE (blue) / — (dim)
- Real-time updates trigger row refresh and gold pulse animation
- Mobile responsive at 375px width

Visual fidelity

- Typography matches: Fraunces serif headlines, Inter body, JetBrains Mono data
- Colors match CSS variables from admin-dashboard-v3.html
- Hover states match (campaign-bar, recipient-row, action-btn)
- Icons match (📧 ⏸️ ❓❓ — ↗)

12. Edge Cases

Plus-one emails

Plus-one recipients get their own approval-plus-one email with their own tracking. Use recipient_type column on email_events to distinguish. Both contribute to the host registration row.

Webhook delays and ordering

Resend may deliver events out of order or up to 5 min late. Aggregate counters are computed via COUNT/MAX/MIN so order does not matter. Forwarded status recomputed on every webhook event.

Duplicate webhooks

Resend may deliver same event twice. The unique index on (resend_email_id, event_type, occurred_at) prevents double-counting. Handle conflict with ON CONFLICT DO NOTHING.

Privacy / GDPR

IP addresses and user agents stored. Document in privacy policy. For Vol. II Houston audience, this is acceptable. For European audiences in Vol. III+, consider IP anonymization.

Email tracking pixel auto-load

Some email clients (Apple Mail with Mail Privacy Protection) auto-load tracking pixels even before the user opens the email. This inflates open counts. Mitigation: use unique_devices and timing patterns to estimate true engagement (Phase 2).

13. Effort Estimate

Task	Time	Risk
Database migration + 2 views	20 min	Low
resend-webhook Edge Function (with svix)	70 min	Medium
Update send-registration-email tags	15 min	Low
Admin: top stats strip	20 min	Low
Admin: campaign accordion + click-to-expand	60 min	Medium
Admin: recipient table with all 10	90 min	Medium

columns		
Admin: filter pills (client-side)	30 min	Low
Admin: action buttons (VIEW/NUDGE/FIX EMAIL)	30 min	Low
Realtime subscription + pulse animation	40 min	Medium
Resend webhook setup + signature test	20 min	Low
Mobile responsive QA	30 min	Low
End-to-end test	30 min	Low
TOTAL	~7.5 hours	—

14. Prompt for Cursor

Once this PRD is reviewed, paste this single message into Cursor chat AND attach both files: the PRD PDF and admin-dashboard-v3.html as visual reference.

> **PASTE INTO CURSOR** Build the email engagement dashboard per attached PRD v2.0 (FINAL). The file admin-dashboard-v3.html is the approved visual mockup — match it pixel-by-pixel for typography, colors, spacing, hover states, and animations. Execute all 10 deployment steps in Section 10 in order. Implement all acceptance criteria in Section 11. Do NOT redesign the UI. Do not break existing functionality (registrations, approvals, check-in). After deployment, post a summary including: (1) Resend webhook URL to paste into Resend dashboard, (2) RESEND_WEBHOOK_SECRET env var instructions, (3) test results from Section 11. If any step fails, stop and report the exact error before continuing.

END OF DOCUMENT

The Horeca Meetup · Houston Vol. II · May 5, 2026